

STICKFRAME

ANIMATION OPERATING SYSTEM — ENGINEERING BLUEPRINT V3

Principal Research Engineer | Internal Design Document | July 2026

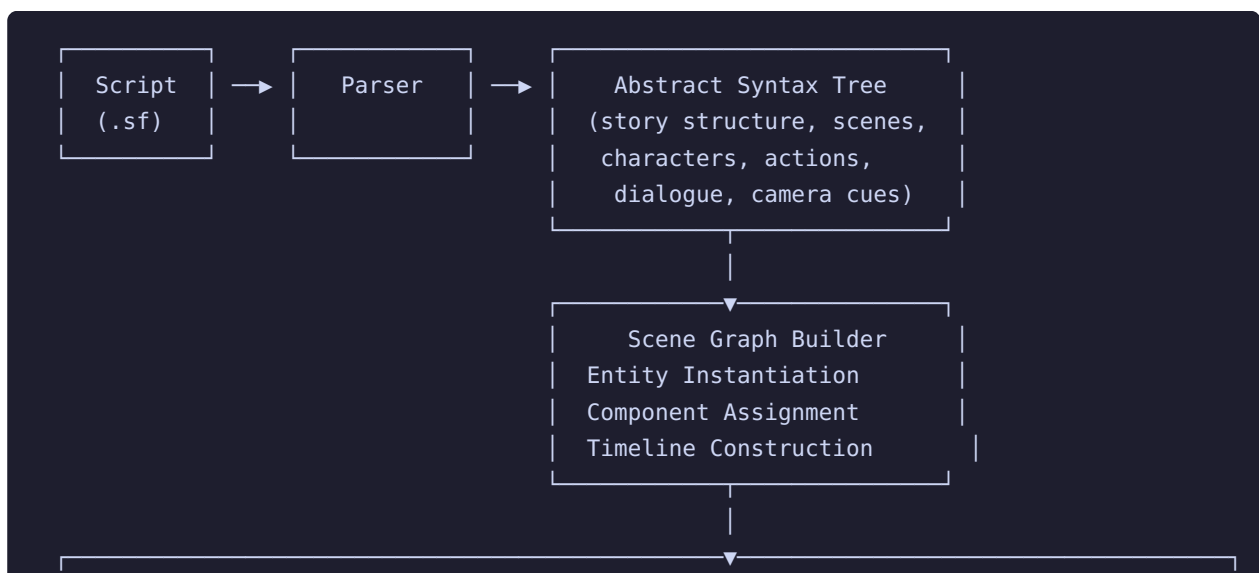
CONFIDENTIAL — ENGINEERING INTERNAL

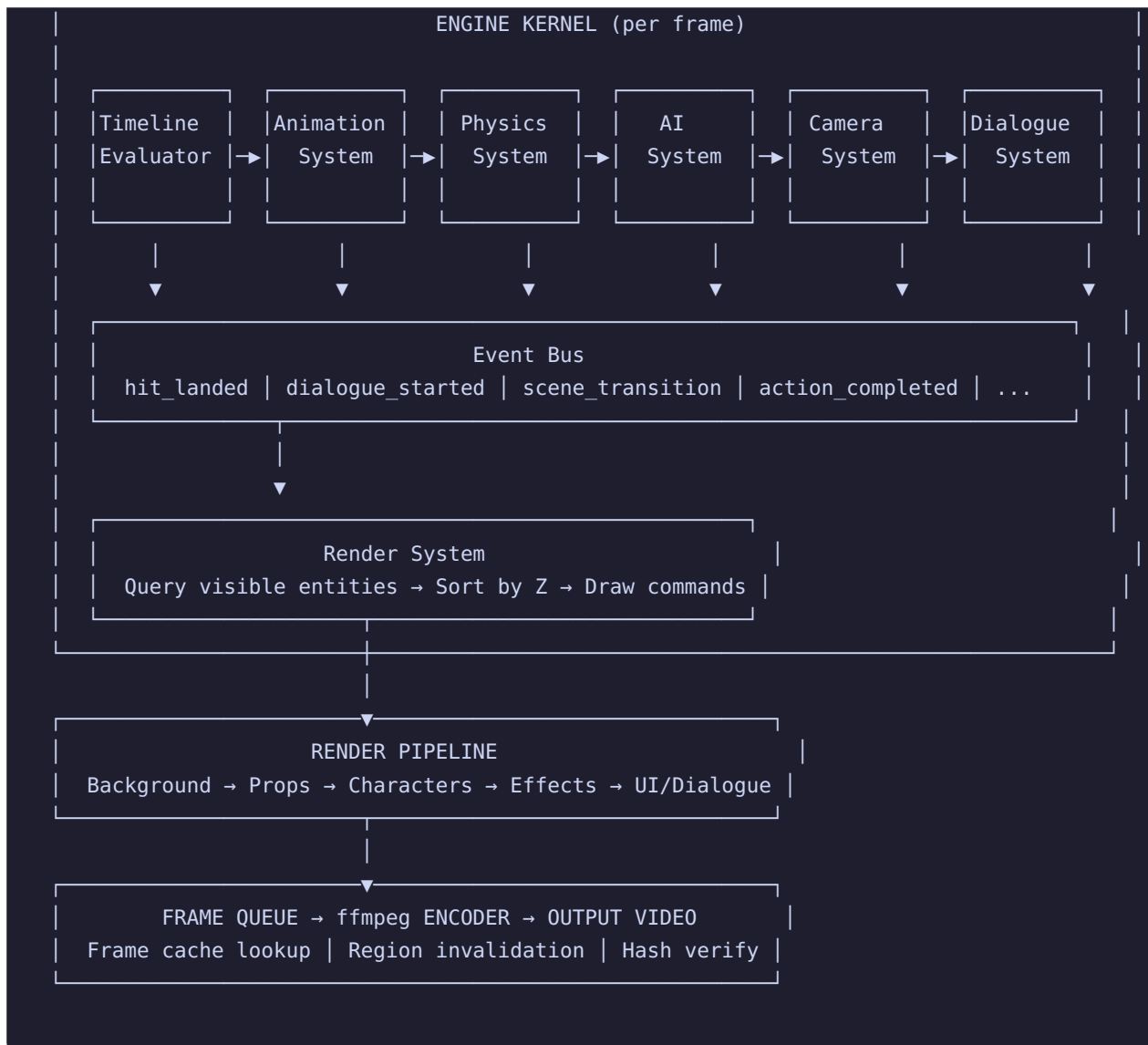
Table of Contents 1. Vision 2. Engineering Philosophy 3. Core Principles 4. Product Scope 5. Functional Requirements 6. Non-Functional Requirements 7. System Architecture 8. Engine Architecture 9. Rendering Pipeline 10. Scene Graph 11. World State Management 12. Character System 13. Physics System 14. Camera System 15. Timeline System 16. Dialogue System 17. Story Understanding Layer 18. AI Director Architecture 19. Rendering Infrastructure 20. Performance Strategy 21. Storage Architecture 22. Networking 23. Security 24. Plugin Architecture 25. API Strategy 26. Testing Strategy 27. Scalability 28. Risks 29. Research Areas 30. Five-Year Engineering Roadmap

1. Vision

7.1 Engine Kernel Data Flow

This diagram represents the complete data flow through the engine kernel — from user input to final video output. It is the single picture every engineer should internalize before writing a line of code.





StickFrame is a programmable animation operating system. It transforms structured stories, scripts, and eventually natural language into complete stickman-animated movies with deterministic, reproducible output.

The system is not an AI video generator. AI is a component, not the foundation. The core is a deterministic engine that takes a well-defined scene description and produces a video. AI layers sit above this core to assist with story breakdown, cinematography decisions, and character direction — but the rendering pipeline itself is fully deterministic.

StickFrame must scale from a 10-second social media clip to a feature-length movie (90+ minutes) within the same architecture. A short clip and a feature film use the same engine, same data model, same pipeline — only the scale of the input differs.

Platform Characteristics:

- Deterministic core engine — same input always produces identical output
- AI as augmentation, not replacement — story understanding and director guidance are optional layers

- Programmable at every level — from the scene DSL to the rendering plugin API
- Distributed by design — rendering scales horizontally across worker nodes
- Character consistency across any project length — persistent rigs, appearance, and behavior definitions

2. Engineering Philosophy

2.1 Determinism First

Frame rendering must be deterministic. Given identical scene input (scripts, character definitions, timeline, random seed), the engine must produce byte-identical output. This enables caching, incremental rendering, regression testing, and distributed rendering where any node can render any frame independently.

2.2 Separation of Concerns

The system is organized into distinct layers: story understanding → scene planning → deterministic engine → rendering. Each layer communicates through well-defined interfaces. The deterministic engine never depends on the story understanding layer. The story layer can be replaced, upgraded, or removed without affecting the rendering pipeline.

2.3 Progressive Complexity

New users produce output with minimal input (single sentence → 30-second video). Power users access the full system: custom rigs, physics overrides, per-scene camera scripts, plugin-defined rendering effects. The same architecture serves both cases — the simple path simply uses sensible defaults at every layer.

2.4 Composition Over Inheritance

The scene graph uses an Entity-Component-System (ECS) architecture rather than deep class hierarchies. Characters, props, cameras, and lights are entities with attached components (TransformComponent, SpriteComponent, AnimationComponent, PhysicsComponent). Systems operate on entities that possess the required component set. This enables the animation equivalent of Blender's node system — composable behavior without rigid class trees.

2.5 Format Agnostic Storage

All project data is stored in a versioned, schema-less document format. The internal representation is never tied to a specific file format. Export plugins convert to MP4, GIF, PNG sequence, or custom formats. Import plugins ingest from Pivot .stk, Spine .json, DragonBones, and custom sources.

3. Core Principles

1. **Deterministic Pipeline.** Every stage from scene parsing to frame output is deterministic. Nondeterministic components (AI, random generation) are isolated in the story understanding layer and produce deterministic intermediate output.
2. **Operator at Every Level.** Users and plugins can insert operators at any pipeline stage: pre-processing scripts, post-processing effects, custom physics solvers, alternative renderers. The pipeline is a directed acyclic graph of operators.
3. **Cache Invalidation Is Explicit.** Each frame's dependency graph is tracked. Changing a character's color invalidates only frames containing that character. Changing a physics parameter invalidates only affected simulation frames.
4. **Offline-First Architecture.** The engine runs entirely offline. Cloud features (collaboration, distributed rendering, asset marketplace) are optional network layers on top of a local-first core.
5. **Backward Compatibility.** Projects created in v1.0 must render identically in v5.0. The scene document format includes a version field and explicit upgrade paths are never silently applied.
6. **Measured Before Optimized.** No optimization without profiling. The engine includes built-in instrumentation at every pipeline stage. Frame render times, memory usage, cache hit rates, and system latencies are always measurable.
7. **Fail Verbosely.** Every validation failure produces a machine-readable error with the exact location and reason. Debug mode outputs per-frame intermediate state as JSON for inspection.

4. Product Scope

4.1 In Scope

- Structured script to stickman animation (deterministic engine)
- Multiple simultaneous stickman characters with distinct rigs and appearances
- Action library: walk, run, jump, attack, fall, idle, wave, sit, point, climb, swim, fly, dance, fight, dodge, block, roll, hug, cry, laugh, bow, spin — extensible via custom action definitions
- Scene composition: backgrounds, props, weather effects, lighting overlays
- Camera system: pan, zoom, follow, shake, split-screen, multi-scene
- Dialogue: speech bubbles, subtitle tracks, TTS voiceover
- Audio: SFX library, background music, per-action sound attachments
- Export: MP4, GIF, PNG sequence, vertical format (9:16 for Shorts/Reels)
- Plugin API for custom rendering effects, importers, exporters, physics solvers
- Distributed rendering across worker nodes

4.2 Out of Scope (v1-v2)

- Real-time interactive editing (focus is batch rendering from script)
- 3D rendering (stickman is a 2D abstraction; 3D is a separate project)
- Full non-linear video editor (external NLE integration via plugin API)
- Procedural voice acting (TTS integration is via plugin)
- User-generated marketplace (post-v2 community feature)

5. Functional Requirements

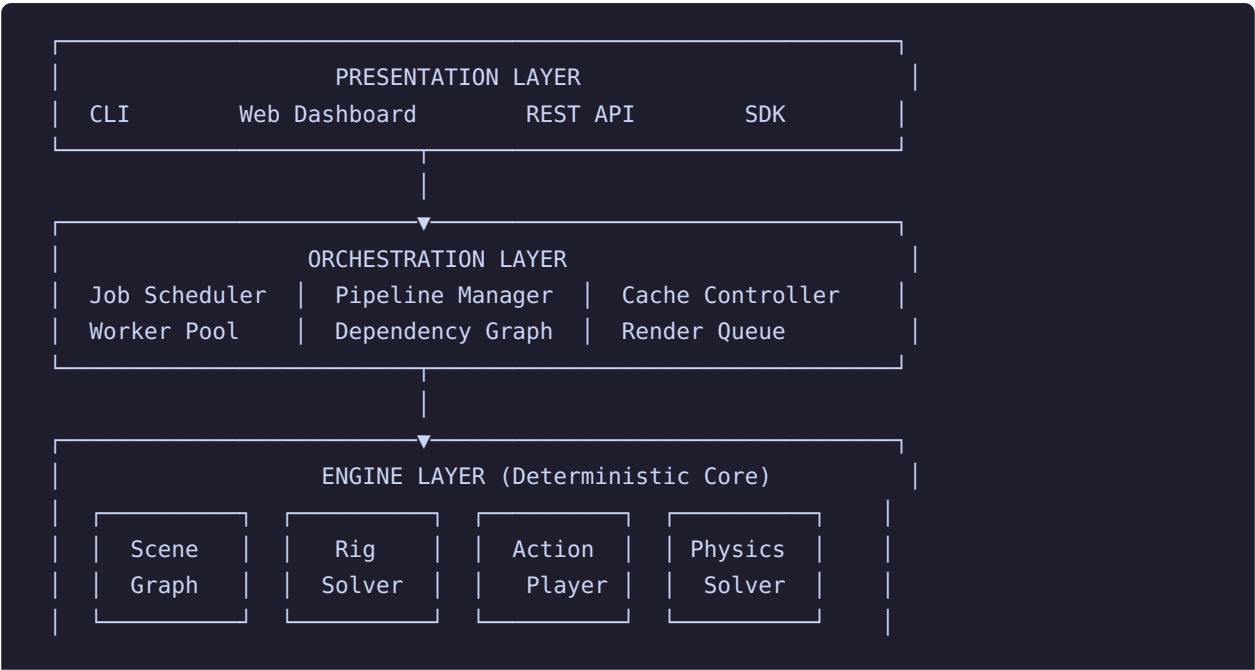
ID	Requirement	Priority
FR-01	The engine shall accept a structured scene description (JSON or DSL) and produce an MP4 video without any human intervention.	P0
FR-02	The engine shall support at least 50 simultaneous characters in a single scene.	P1
FR-03	The action library shall contain at least 20 built-in actions, each defined by skeletal keyframes.	P0
FR-04	The engine shall support user-defined actions via a JSON keyframe format.	P1
FR-05	The timeline shall support event scheduling at sub-frame precision.	P0
FR-06	The camera shall support keyframe-based movement with tweening.	P1
FR-07	The engine shall support scene transitions (cut, fade, crossfade, wipe).	P1
FR-08	Dialogue shall support speech bubble rendering with configurable bubble styles.	P2
FR-09	The engine shall support TTS voiceover via a plugin provider interface.	P1
FR-10	Rendering shall be reproducible: same input + seed = byte-identical output.	P0
FR-11	The engine shall support distributed rendering where frames are divided across worker nodes.	P1
FR-12	The plugin system shall support custom render passes, importers, exporters, and physics solvers.	P1
FR-13	The system shall provide a REST API for job submission, status polling, and result retrieval.	P2
FR-14	The project format shall be forward-compatible: projects openable by newer versions.	P0

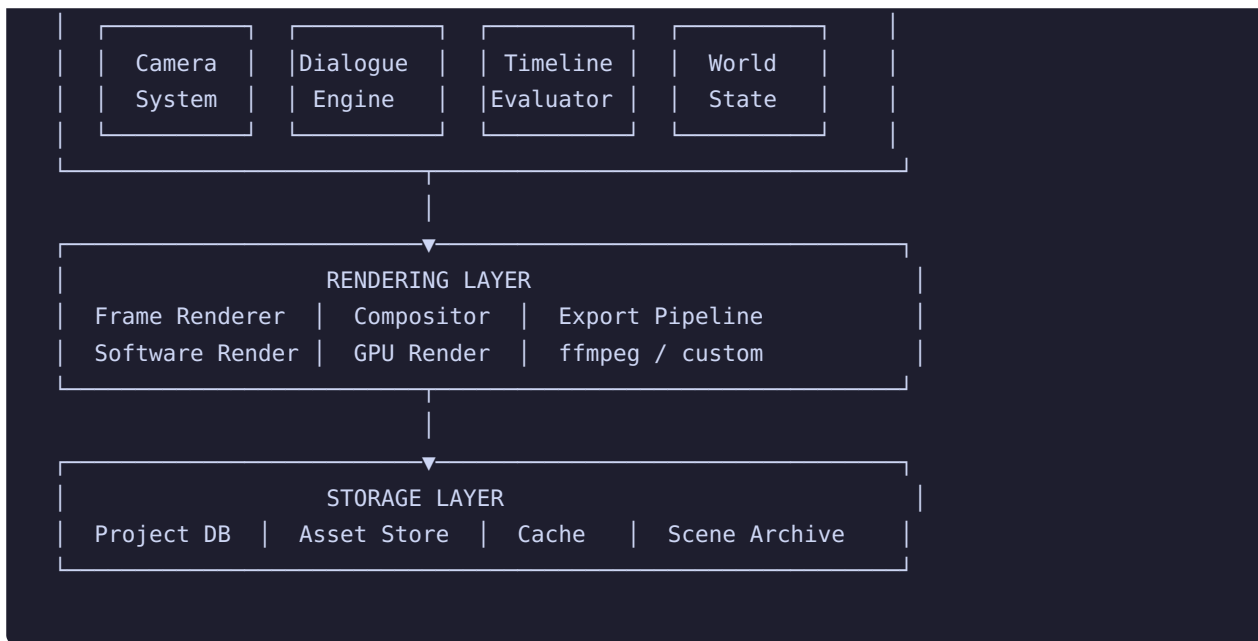
6. Non-Functional Requirements

ID	Requirement	Target
NFR-01	Frame render latency (single character, no effects)	< 10 ms per frame (CPU-only)
NFR-02	Maximum scene duration (single project)	Unlimited (tested to 3 hours)
NFR-03	Maximum characters per scene	200+ (bottleneck is available RAM)
NFR-04	Deterministic output guarantee	100% — across OS, Python version, hardware
NFR-05	Cache hit rate (incremental renders)	Target > 90% for single-character changes
NFR-06	Maximum project file size	Unlimited (streaming document model)
NFR-07	Memory per worker process	Configurable ceiling (default 512 MB)
NFR-08	Plugin isolation	Plugins cannot crash the engine (sandboxed)
NFR-09	API response time (job submission)	< 100 ms
NFR-10	Cross-platform	Linux (primary), macOS, Windows
NFR-11	Backward compatibility	All scene files from v1.0 openable in v5.0

7. System Architecture

The system is organized into five architectural layers, each with defined boundaries and interfaces:





Orchestration layer manages job lifecycle: parsing input, building dependency graphs, dispatching frames to workers, collecting results, and managing caches. It does no rendering work itself.

Engine layer is the deterministic core. It takes a fully resolved scene and produces a list of per-frame draw commands. No randomness. No external calls. Everything is computed from the scene state alone.

Rendering layer converts draw commands into pixels. It supports a software renderer (Pillow-based, deterministic) and will support a GPU-accelerated renderer via another plugin. The rendering layer is stateless — it receives a frame's draw commands and returns an image.

8. Engine Architecture

The engine follows a fixed-timestep update loop inspired by game engine architecture (Godot, Unity):

```
For each frame at time T:
  1. Scene Graph Traversal – build list of visible entities with transforms
  2. Timeline Evaluation – resolve which actions are active at T
  3. Action Player Update – advance each character's animation by dt
  4. Rig Solver – compute joint positions from current keyframe
  5. Physics Tick – advance physics simulation by fixed dt
  6. Camera System – compute viewport transform from camera keyframes
  7. Dialogue Engine – resolve speech bubble state at T
  8. Draw Command Generation – produce sorted list of draw commands
  9. Submit to Render Queue
```

The engine operates on a **fixed internal timestep** (default 60 Hz) independent of the output frame rate. Rendering can output at any integer divisor (30 fps, 24 fps, 12 fps) by sampling the internal state at the appropriate interval.

Systems within the engine are ordered and deterministic. The system execution order is fixed per engine version. Each system receives the world state and returns a diff. Systems are pure functions of (input state, delta time) → output state plus draw commands.

Reference Architecture: Godot Engine's scene system (nodes as composable elements, not inheritance) and Bevy Engine's ECS architecture (systems operating on query results from component tables). The StickFrame engine is simpler — it does not need to handle real-time input, but it must handle far longer durations (hours vs minutes).

9. Entity-Component System

The engine uses an Entity-Component System (ECS) architecture as its core data model, inspired by Bevy Engine, Unity DOTS, and Godot's scene system. ECS provides the scalability, composability, and cache efficiency needed for scenes with hundreds of entities and dozens of concurrent systems.

9.1 Data Model

```
World:
  entities: Vec
  components: SparseSet<
  resources: HashMap

Entity: u64 (generational index)
  - generation: u32 (tracks entity lifetime for safe ID reuse)
  - index: u32 (dense array index)

Component: Pod (plain old data struct)
  - Position { x, y, z }
  - Velocity { vx, vy }
  - Transform { parent: Option, local_x, local_y, rotation, scale }
  - AnimationPlayer { current_action, frame, speed, blend_state }
  - Skeleton { bone_angles: [f32; N], bone_lengths: [f32; N] }
  - PhysicsBody { mass, velocity, collision_shape, flags }
  - Appearance { head_radius, limb_thickness, colors }
  - DialogueState { active_text, bubble_style, progress }
  - Renderable { z_order, visible, draw_commands_cache }
  - Health { current, max }
  - AI { behavior_tree, state }
  - Camera { viewport, target_entity, zoom, shake }

Resource (singleton):
```

```

- Timeline { tracks, current_time }
- CameraState { active_camera, viewport_transform }
- GlobalSettings { seed, fps, resolution }
- EventQueue { pending_events }

```

9.2 System Execution Order

Systems are pure functions that query entities by component signature and produce state diffs. System order is fixed per engine version and explicitly declared:

Frame Tick:

1. InputSystem – read timeline events for this frame
2. AnimationSystem – advance animation players, compute bone angles
3. PhysicsSystem – integrate velocities, detect collisions
4. AISystem – evaluate behavior trees, generate actions
5. CameraSystem – compute viewport transform
6. DialogueSystem – resolve speech bubble states
7. EventSystem – process event bus, trigger reactions
8. RenderSystem – generate draw commands from visible entities
9. CleanupSystem – remove dead entities, flush events

9.3 Query API

```

// Query all entities with Position + Velocity
for (entity, (pos, vel)) in world.query::<(Position, Velocity)>() {
    pos.x += vel.vx * dt;
    pos.y += vel.vy * dt;
}

// Query with exclusion
for entity in world.query::<(Position,>().without::() {
    // Entities without physics – animation-driven movement
}

```

The query API is the only way systems access component data. Systems never hold direct references to components — they receive iterators from the ECS scheduler. This enables parallel system execution (two systems targeting disjoint component sets run simultaneously).

9.4 Why ECS Over OOP

Concern	OOP (class hierarchy)	ECS (component composition)
Add jumping ability	Create new subclass	Add JumpComponent

Character becomes ghost	Refactor entire hierarchy	Remove PhysicsBody, add GhostComponent
1000 characters	Deep vtable overhead	Dense component arrays, cache-friendly
Custom behavior	Inheritance hell	Compose existing components + new system

ECS makes the engine fundamentally extensible. A plugin adding weather effects simply introduces a `WeatherComponent` and `WeatherSystem` — no existing code changes.

10. Event Bus

The event bus decouples systems by providing a publish-subscribe communication channel. Systems emit events; other systems subscribe. No system holds a direct reference to another system.

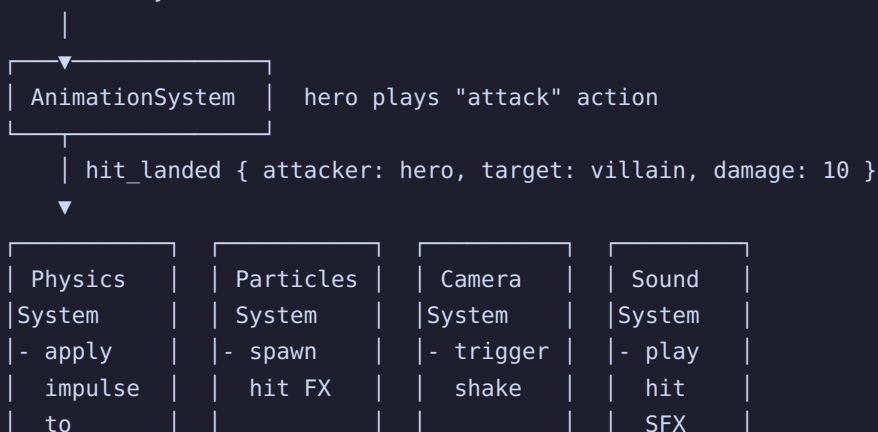
10.1 Event Types

Event:

- entity_created { entity_id, template }
- entity_destroyed { entity_id }
- action_started { entity_id, action, params }
- action_completed { entity_id, action }
- collision { entity_a, entity_b, contact_point, impulse }
- dialogue_started { entity_id, text }
- dialogue_completed { entity_id }
- camera_shake { intensity, duration }
- hit_landed { attacker, target, damage }
- scene_transition { from_scene, to_scene, transition_type }
- custom { type: string, payload: json }

10.2 Event Flow Example

Timeline says: hero attacks villain at frame 120



villain

Systems never call each other. The AnimationSystem emits `hit_landed`. Any system subscribed to that event reacts independently. Adding a new reaction (e.g., screen flash on hit) requires zero changes to the AnimationSystem.

10.3 Event Queue

Events are queued during the frame and processed in a dedicated EventSystem pass at the end of the frame. This prevents cascading event chains within a single frame ($A \rightarrow B \rightarrow C \rightarrow \text{infinite loop}$). Events can be delayed (process at frame $T+N$), batched (merge identical events), or prioritized (critical path events process first).

10.4 Serialization

All events are serializable to JSON. This enables event recording for debugging ("what happened during these 100 frames?"), event replay for regression testing, and networking (remote workers subscribe to events from the orchestrator).

9. Rendering Pipeline

The rendering pipeline converts per-frame draw commands into pixels. The pipeline is a fixed sequence of stages, each of which can be extended via plugins:

Draw Commands (sorted by Z-order)

Stage 1: Background Rendering

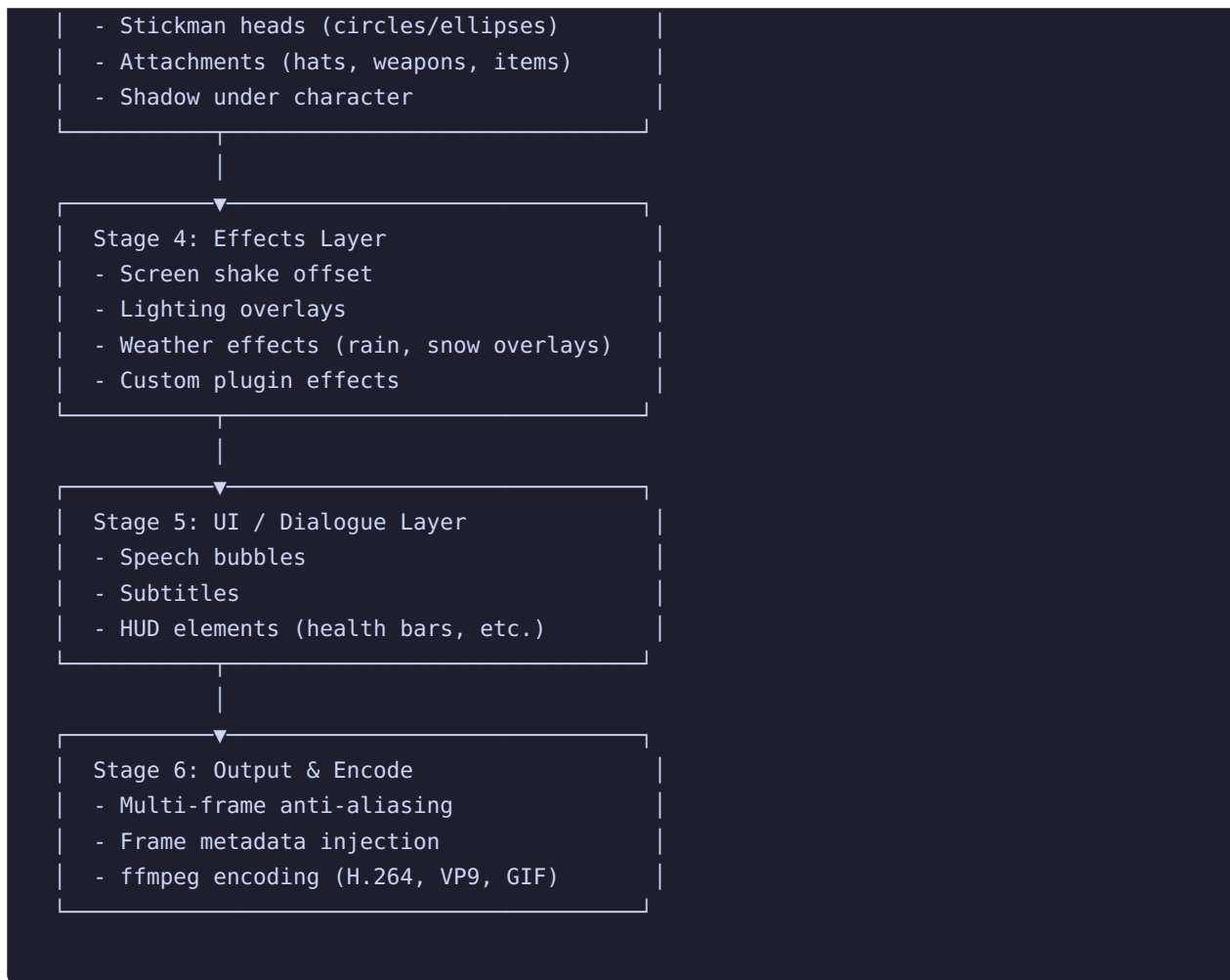
- Solid color fills
- Gradient backgrounds
- Image backgrounds (with parallax)

Stage 2: Object Rendering

- Static props (trees, buildings, etc.)
- Particle systems (rain, smoke, FX)

Stage 3: Character Rendering

- Stickman limbs (line segments)



Software renderer (default): Uses Pillow as the canvas. Each draw command is resolved to Pillow operations (line, ellipse, polygon, text). The renderer maintains a pixel buffer per frame and composites layers in Z-order.

GPU renderer (future): Draw commands are translated to OpenGL/Vulkan draw calls. The same command format serves both renderers. The choice of renderer does not affect output — the same scene should produce perceptually identical results.

10. Scene Graph

The scene graph is an Entity-Component tree. Every entity has a Transform component that defines its position, rotation, and scale relative to its parent. The tree structure enables hierarchical animation — moving a parent entity moves all children.

```
Scene (root)
├── Camera
│   └── CameraConfig (FOV, zoom, clipping)
├── Environment
│   └── Background
```

```

|   |   └─ SkyGradient
|   |   └─ ParallaxLayer[0..N]
|   └─ AmbientLight
└─ Character[0..N]
    └─ Transform (position, rotation, scale)
    └─ Appearance (head shape, colors, limb proportions)
    └─ AnimationPlayer (current action, frame, blend state)
    └─ SkeletonRig (joint angles, bone lengths)
    └─ PhysicsBody (if physics-enabled)
    └─ Attachment[0..N] (weapons, hats, held items)
        └─ Transform (relative to attach point)
└─ Prop[0..N]
    └─ Transform
    └─ Shape (rectangle, circle, polygon, image)
    └─ PhysicsBody (optional)
    └─ AnimationPlayer (if animated prop)
└─ ParticleEmitter[0..N]
    └─ Transform
    └─ ParticleConfig (rate, lifetime, speed, color)
└─ Timeline
    └─ Track[0..N] (parallel event sequences)
    └─ GlobalTime

```

Design Decision: Scene graph uses a flat ID-based entity lookup (hash map) for $O(1)$ entity access, with a separate hierarchical tree for transform resolution. This split enables fast entity queries for systems while maintaining parent-child semantics for transforms. Godot Engine uses a similar approach with NodePaths vs direct references.

11. World State Management

World state is the authoritative representation of everything in the scene at a given point in time. It is a snapshot of all entity component data. The world state is immutable — each frame produces a new world state from the previous state plus the delta-time update.

State structure:

```
WorldState:
  frame_number: int
  time: float (seconds)
  seed: int (for deterministic RNG)
  entities: { entity_id: ComponentBag }
  scene_graph: adjacency list (parent-child relationships)
  camera state: { position, zoom, target }
```

```
global_flags: { dialogue_active, scene_transitioning, ... }
```

State query system: Systems query state using a query language:

`entities_with(TransformComponent, AnimationComponent)`. This is the ECS pattern. The query result is a read-only view. Systems produce diffs that are applied atomically to produce the next state.

Deterministic RNG: Each frame has a deterministic seed derived from the project seed + frame number. Any system that needs random values (particle systems, camera shake) uses this seeded RNG. This guarantees reproducibility.

Dirty region tracking: When a component changes, the system records which entities and components were affected. This feeds the frame cache — unchanged entities reuse cached draw commands from the previous frame.

12. Character System

12.1 Skeleton Rig

The stickman skeleton is a hierarchical bone structure. Each bone has an angle relative to its parent bone. The root bone (body center) is positioned at the character's world position.

```
Bone Hierarchy (relative to parent):
body (root, world position)
  └─ head (offset from body top)
  └─ left_arm.upper (from shoulder)
    └─ left_arm.lower (from elbow)
  └─ right_arm.upper
    └─ right_arm.lower
  └─ left_leg.upper (from hip)
    └─ left_leg.lower (from knee)
  └─ right_leg.upper
    └─ right_leg.lower
```

12.2 Appearance

Appearance is a component, not a property of the character class. This allows multiple characters to share a rig but have different appearances, and for appearance to be animated independently of the skeleton (glow effects, color cycling).

Appearance properties: `head_radius`, `body_length`, `limb_thickness`, `segment_colors` (per bone group), `head_color`, `eye_style`, `attachment_points` (named positions for accessories).

12.3 Action Definitions

Each action is a named sequence of keyframes. Each keyframe specifies bone angles at a point in time. Actions are defined independently of the skeleton — the same "walk" action works on any bipedal rig with compatible bone names.

```
Action "walk":
  duration: 0.8s (at 1.0x speed)
  keyframes:
    - time: 0.0
      bones: { left_arm.upper: 30°, right_leg.upper: -25°, ... }
      easing: linear
    - time: 0.25
      bones: { left_arm.upper: 0°, right_leg.upper: 0°, ... }
      easing: ease_in_out
    - time: 0.5
      bones: { left_arm.upper: -30°, right_leg.upper: 25°, ... }
      easing: linear
    - time: 0.75
      bones: { ... }
      easing: ease_in_out
  loop: true
  speed_multiplier: 1.0
```

12.4 Action Blending

When transitioning between actions (e.g., walk → jump), the engine cross-fades the bone angles over a configurable blend duration. The blend uses spherical linear interpolation (slerp) for angles to avoid gimbal lock artifacts. Multiple actions can layer — upper body attacks while lower body walks.

Action layering uses a priority system. Higher-priority actions override specific bone groups. A punch action (priority 10) overrides arm bones while the walk action (priority 0) continues to animate leg bones beneath it.

13. Physics System

The physics system provides optional physical simulation for characters and props. It is a deterministic 2D physics engine, not a general-purpose physics library wrapper. This ensures deterministic output independent of the underlying physics library.

13.1 Scope

- Gravity simulation for falling objects and characters
- Collision detection between characters and props (axis-aligned bounding boxes)

- Simple ragdoll physics (joint constraints with angular limits)
- Projectile motion (thrown objects, arrows)
- Platform collision (characters standing on surfaces)

13.2 Design

Physics entities have a `PhysicsBody` component with mass, velocity, collision shape (rectangle, circle), and physics flags (`gravity_enabled`, `collisions_enabled`, `kinematic`). The physics system runs at a fixed sub-step (default 120 Hz) independent of the render frame rate, with configurable sub-step count for stability.

Physics is optional. A character without a `PhysicsBody` component uses its animation keyframes directly — the root position is determined by the animation, not physics. This enables hybrid scenes where some characters follow scripted paths and others are physically simulated.

Constraint: Physics introduces potential nondeterminism due to floating-point accumulation differences across platforms. Mitigation: use fixed-point arithmetic for physics simulation, or a strict cross-platform math library. The physics engine includes a validation step: after each sub-step, the total energy of the system is checked against a tolerance. Energy drift triggers a warning but does not abort rendering.

14. Camera System

The camera determines the viewport transform applied to all scene elements before rendering. The camera system supports multiple camera definitions within a scene, with the active camera switching at configurable times.

14.1 Camera Properties

- **Viewport:** width, height, aspect ratio (16:9, 9:16, 1:1, custom)
- **Position:** offset from scene origin (x, y)
- **Zoom:** scale factor (1.0 = default, 2.0 = 2x zoom)
- **Rotation:** viewport rotation (degrees)
- **Target:** optional entity ID for follow camera
- **Follow offset:** (x, y) offset from tracked entity
- **Follow smoothness:** lerp factor for camera catch-up

14.2 Camera Keyframes

Camera movement is keyframe-able, same as character actions. Camera keyframes interpolate position, zoom, and rotation. Easing functions apply. Multiple camera tracks can exist in parallel — the active track is selected by the timeline.

14.3 Special Camera Modes

- **Shake:** Perlin-noise-based screen offset with configurable intensity and frequency
 - **Split screen:** Divide viewport into regions, each with an independent camera
 - **Picture-in-picture:** Small second viewport overlaid on main viewport
 - **Wide shot:** Zoom out to show entire scene (auto-calculate zoom from entity bounding box)
 - **Dynamic framing:** Auto-adjust zoom to keep all characters visible
-

15. Timeline System

The timeline is the central coordination mechanism. It describes what happens, when, and to whom. The timeline is a sequence of parallel and serial event tracks.

15.1 Timeline Structure

```
Timeline:
  metadata: { fps, duration, project_version }
  tracks:
    - track_id: "character_hero"
      type: action_sequence
      events:
        - time: 0.0
          action: "idle"
          params: {}
        - time: 2.0
          action: "walk"
          params: { direction: "right", speed: 1.0 }
        - time: 4.5
          action: "jump"
          params: { height: 100, distance: 150 }
        - time: 6.0
          action: "attack"
          params: { target: "villain", attack_type: "punch" }

    - track_id: "camera_main"
      type: camera_track
      events:
        - time: 0.0
          target: "hero"
          zoom: 1.0
        - time: 3.0
          target: "hero+villain"
          zoom: 0.7
        - time: 5.0
          shake: { intensity: 0.5, duration: 0.3 }

    - track_id: "dialogue_hero"
```

```

type: dialogue_track
events:
  - time: 1.0
    text: "You'll never get away with this!"
    style: angry
  - time: 4.0
    text: "Ha! Take that!"
    style: happy

- track_id: "scene_transitions"
  type: transition_track
  events:
    - time: 0.0
      type: "cut"
    - time: 10.0
      type: "fade"
      duration: 1.0

```

15.2 Event Types

- **action_sequence**: Ordered list of character actions with timing and parameters
- **camera_track**: Camera keyframes, target changes, effects
- **dialogue_track**: Speech events with text, style, voice
- **transition_track**: Scene change events (cut, fade, wipe)
- **physics_track**: Physics state changes (gravity toggle, impulse forces)
- **prop_track**: Prop appearance, movement, transformation
- **custom_track**: Plugin-defined track types

15.3 Evaluation

The timeline evaluator walks all tracks in parallel, sorted by time. At each frame, it determines which events are active and produces the action commands for that frame. Events can be one-shot, looping, or triggered by other events (when character A reaches position X, trigger event Y).

The timeline is **not** a real-time media timeline. It is a constraint system. Events define constraints on the world state. The engine resolves the constraint set into the minimal set of action commands needed to produce a valid state at each frame.

16. Dialogue System

The dialogue system handles text display, timing, and voiceover integration. It operates as a rendering stage after character and camera systems.

16.1 Speech Bubbles

- Bubble position: anchored to character's head position (configurable offset)
- Bubble style: configurable per character (tail position, border width, background color, font)
- Text wrapping: auto-wrap within bubble width with configurable max width
- Typing animation: characters appear one-by-one at configurable speed
- Bubble visibility: fade-in/fade-out with configurable lead/lag times

16.2 Subtitles

The subtitle track is generated from the same dialogue data. Subtitles are rendered at the bottom of the viewport with optional drop shadow. Subtitle timing is exact to the frame (subtitle appears on the frame where dialogue starts).

16.3 Voiceover

Voiceover is handled through a provider interface. The engine calls

`tts_provider.synthesize(text, voice_id) → audio_buffer` and attaches the result to the output video. Multiple providers can be registered (system TTS, edge-tts, ElevenLabs, custom). The voiceover call is the only part of the rendering pipeline that may be nondeterministic — it is isolated from the frame rendering path. The video renders first; audio is mixed in as a post-process.

16.4 Lip Movement (Future)

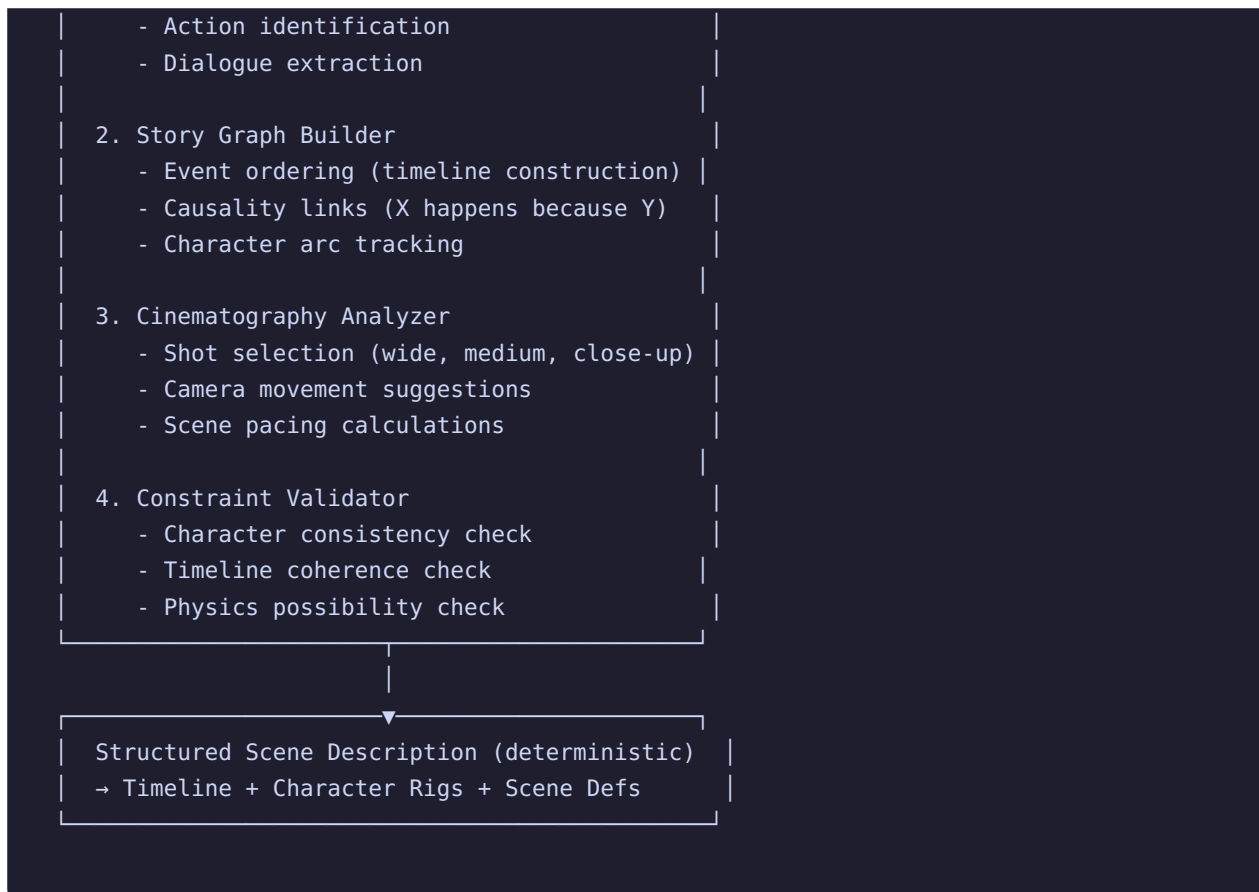
For dialogue without voiceover, characters can show mouth movement synced to dialogue timing. A simple open-close animation modulated by syllable timing. The syllable timing is derived from the dialogue text using a phoneme estimation algorithm.

17. Story Understanding Layer

The story understanding layer (SUL) converts natural language or loosely structured input into the deterministic scene description that the engine consumes. This is the only non-deterministic layer. The SUL is replaceable and optional.

17.1 Architecture





17.2 Story Understanding Approaches (Research)

Approach A: LLM-based (near-term). Use a large language model to parse input. The LLM receives a structured prompt with the allowed actions, character properties, and scene elements. It outputs JSON conforming to the timeline schema. This approach is fast to implement but nondeterministic — different LLM runs may produce different outputs. The user can lock a specific output by saving the LLM response as the scene description.

Approach B: Constraint-based (long-term). Implement a story grammar system. The system defines valid story structures as a grammar. User input is parsed into the grammar, producing a story graph. The story graph is then compiled into the timeline. This approach is deterministic but requires significant linguistic engineering.

Approach C: Hybrid. Use an LLM for initial parsing and story graph construction, then validate the output against a constraint system. The constraint system catches impossible timelines (character speaking after death, simultaneous contradictory actions, physics violations). The validated output is deterministic.

Recommendation: Start with Approach A (LLM-based) for rapid development. Research Approach C (hybrid with constraint validation) for Year 2-3. The SUL output format (structured scene description) is independent of the parsing method — the engine never knows whether the input came from AI, a human writer, or a script file.

18. AI Director Layer (Future)

The AI Director Layer is an autonomous cinematography system that sits above the story understanding layer. It is architecturally separate from the engine kernel — the engine never depends on it. A dedicated architecture document exists at `research/docs/architecture-ideas/ai_director_architecture.md` covering the full subsystem design.

This section provides a summary; the complete specification (including story parser, scene planner, character planner, camera planner, choreographer, dialogue planner, and validator subsystems) lives in the dedicated document.

The AI Director sits above the story understanding layer and acts as an autonomous cinematographer. It makes real-time decisions about camera placement, shot composition, scene pacing, and action selection based on story context.

18.1 Director Responsibilities

- **Shot selection:** Choose camera angle and distance based on scene emotional content
- **Pacing:** Adjust scene duration based on narrative tension curve
- **Action suggestion:** Propose character actions that advance the story (not explicitly written)
- **Dramatic emphasis:** Slow motion, close-ups, camera shake during key moments
- **Character blocking:** Suggest character movement paths for visual composition

18.2 Implementation Strategy

The AI Director is an Agent system operating on the scene graph and timeline before rendering. It produces a "director's cut" modification to the timeline. The original timeline is preserved; director modifications are a separate overlay that can be accepted, rejected, or edited.

The Director uses no generative AI for visual output. It generates structured commands — camera keyframes, action assignments, pacing adjustments — that the deterministic engine renders. This ensures that enabling or disabling the Director does not change the underlying engine behavior.

18.3 Research Questions

- How to define a "film grammar" as a set of rules the Director can follow?
- Can shot selection be modeled as a constraint satisfaction problem?
- What is the minimal set of camera positions that can tell any story?
- How to evaluate Director quality objectively (not through user surveys)?

19. Rendering Infrastructure

The rendering infrastructure is the distributed system that converts the engine's output into final video files. It is designed to scale from a single laptop to a render farm with hundreds of nodes.

19.1 Architecture



19.2 Render Farm Design

StickFrame's distributed rendering is inspired by OpenCue (open-source render farm manager by Sony Pictures Imageworks) and sqi (modern distributed render farm). Each worker is a stateless process that receives a frame range and scene snapshot, renders frames, and returns them to the assembler.

Workers communicate via a message queue (Redis Streams or RabbitMQ). The scheduler tracks frame completion and handles retries. If a worker fails, its frames are reassigned to another worker.

The output assembler collects completed frames, verifies they match the expected hash, and concatenates them with ffmpeg.

19.3 Reference: OpenCue

OpenCue (Apache 2.0, Python) is an open-source render farm manager used in VFX production. Its architecture includes: CueBot (job management), CueGUI (job monitoring), RQD (render queue daemon on each node), and a PostgreSQL-backed database for job state. StickFrame's worker system follows OpenCue's job/frame/layer abstraction but is simplified for 2D rendering (no 3D scene loading, no software licensing).

20. Performance Strategy

Performance targets are defined per frame, per scene, and per project. The engine is designed for predictable performance — worst-case render time should be estimable from input size.

20.1 Frame-Level Performance

Operation	Target (single char, no effects)	Budget
Scene graph traversal	0.1 ms	< 1%
Timeline evaluation	0.05 ms	< 0.5%
Animation interpolation	0.02 ms per character	< 0.2%
Rig solving	0.03 ms per character	< 0.3%
Physics simulation	0.5 ms (fixed step)	~5%
Draw command generation	0.1 ms	~1%
Pillow rendering (720p)	5-15 ms	~85%
ffmpeg encode (per frame)	< 1 ms (batch amortized)	< 1%

20.2 Optimization Strategies

- **Frame caching:** Cache rendered frames keyed by their dependency hash. If a frame's dependencies haven't changed, return the cached image. This makes incremental edits nearly instant.
- **Draw command batching:** Group draw commands by type and Z-layer to minimize render state changes.
- **Region invalidation:** Only re-render regions of the frame that changed. If only a character's arm moved, only the pixels covered by that character's bounding box need updating.

- **Parallel frame rendering:** Frames are independent — render multiple frames concurrently using a thread pool or multiprocessing pool.
- **Lossy preview mode:** Render at 1/2 or 1/4 resolution during development. The same frame commands work at any resolution.
- **Sub-frame compositing:** Characters on different Z-layers can be rendered in parallel and composited.

20.3 Memory Management

The engine streams frames to disk during rendering rather than holding all frames in memory. For a 90-minute movie at 30 fps (162,000 frames), each 720p PNG is ~200 KB — total 32 GB uncompressed. The engine writes frames to a temporary directory and pipes them into ffmpeg as they complete. The memory footprint stays under 1 GB regardless of total frame count.

21. Storage Architecture

21.1 Project Format (.sfproj)

Each StickFrame project is a directory containing:

```
project.sfproj/
├─ manifest.json          # Project metadata, version, author
├─ scene/                 # Scene definitions
│   ├─ scene_001.json
│   ├─ scene_002.json
│   └─ ...
├─ timeline/              # Timeline files (may reference external scene files)
│   ├─ main.json
│   └─ ...
├─ characters/            # Character rig and appearance definitions
│   ├─ hero.json
│   ├─ villain.json
│   └─ ...
├─ actions/               # Custom action definitions (built-in actions are engine-internal)
│   ├─ custom_dance.json
│   └─ ...
├─ assets/                # External assets (images, audio, fonts)
│   ├─ backgrounds/
│   ├─ sfx/
│   ├─ music/
│   └─ fonts/
├─ plugins/               # Project-specific plugins
│   └─ ...
└─ output/                # Rendered output
    └─ output.mp4
```

The project format is a directory on disk. No binary blobs — all scene data is JSON. Assets are referenced by hash and stored in a content-addressable store. This enables deduplication (same asset used in multiple scenes stored once) and integrity verification (detect corrupted or modified assets).

21.2 Content-Addressable Asset Store

Assets are stored by SHA-256 hash of their content. References use the hash: `"background": "sha256:a1b2c3d4..."`. The asset store maps hashes to files. Multiple scenes referencing the same asset share one copy. The store supports local disk, S3-compatible, and custom backends.

21.3 Caching Layer

The cache stores rendered frames, intermediate computations (solved rig positions, physics simulation results), and compiled action data. The cache is keyed by a hash of input dependencies (scene state + frame number + engine version). Cache entries are invalidated when:

- Any component in the dependency changed
- The engine version changed (cached output may differ)
- Cache TTL expired (configurable, default 30 days)

The cache uses SQLite for metadata (mapping keys to file locations) and disk/S3 for storage.

21. Script DSL

The StickFrame Script Language (.sf) is the primary authoring interface for the platform. It is a declarative, story-focused language designed to be both human-readable and machine-parsable. The DSL is the bridge between the author's intent and the engine's deterministic scene description.

21.1 Design Goals

- **Readable as a script.** A non-technical writer should understand the structure by reading it.
- **Unambiguous.** Every valid script maps to exactly one timeline. No hidden state, no implicit defaults that change behavior silently.
- **Extensible.** The grammar is versioned and extensible. New keywords can be added without breaking existing scripts.
- **Compilable.** The DSL compiles to the internal Timeline + Scene Graph representation. The compiler is the only subsystem that understands the DSL — the engine never sees raw source text.

21.2 Example

```
# StickFrame Script v1.0
# File: duel.sf

scene arena width=1280 height=720 fps=30 bg="#87CEEB"

character hero:
  rig bipedal
  appearance head_color="#FFD700" body_color="#FF6347"
  position (200, 500)

character villain:
  rig bipedal
  appearance head_color="#8B0000" body_color="#4A0000"
  position (600, 500)

camera main:
  follow hero
  zoom 1.0

timeline:
  scene arena:
    0.0s    hero.idle
    0.5s    villain.idle
    1.0s    hero.walk(direction="right", speed=1.5)
    2.5s    hero.speak(text="You'll never win!")
    3.0s    villain.speak(text="We'll see about that...")
    3.5s    villain.walk(direction="left", speed=1.2)
    4.0s    camera shake(intensity=0.5)
    4.5s    hero.attack(target=villain, type="punch")
    5.0s    villain.hit(damage=10)
    5.2s    villain.fall
    6.0s    hero.celebrate
```

21.3 Grammar (BNF Fragment)

```
script      ::= statement*
statement   ::= scene_def | char_def | camera_def | timeline_def | timeline_event

scene_def   ::= "scene" ident ("width=" int)? ("height=" int)? ("fps=" int)? ("bg=" string)?
char_def     ::= "character" ident ":" indent char_property+ dedent
char_property ::= "rig" ident
               | "appearance" (ident "=" value)+
               | "position" "(" number "," number ")"
camera_def   ::= "camera" ident ":" indent camera_property+ dedent
camera_property ::= "follow" ident | "zoom" number

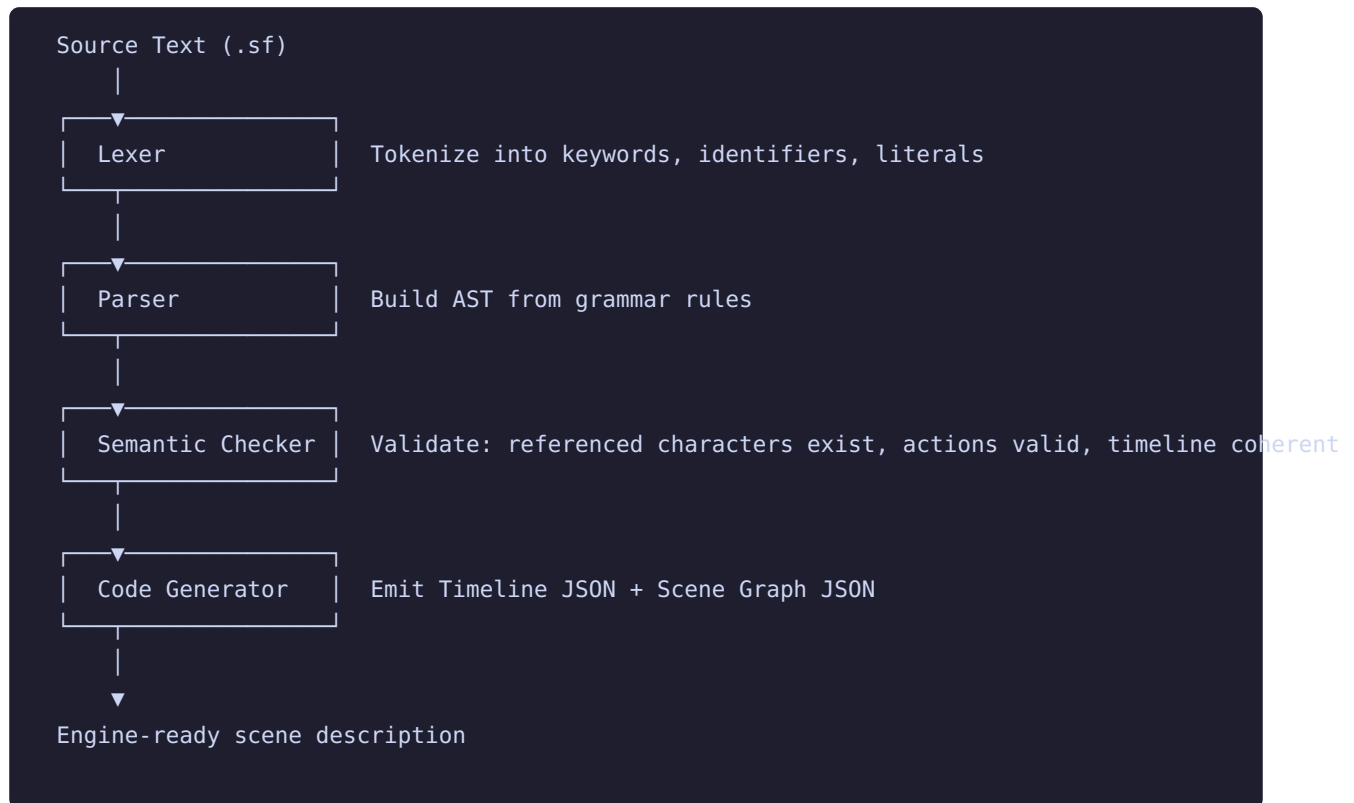
timeline_def ::= "timeline" ":" indent scene_block+ dedent
```

```

scene_block ::= "scene" ident ":" indent timeline_event+ dedent
timeline_event ::= number "s" "." ident "(" param ("," param)* ")"?
param ::= ident "=" (ident | number | string)

```

21.4 Compiler Pipeline



The compiler is deterministic. The same source text always produces the same intermediate representation. This enables version-controlled scripts and diff-based timeline changes.

Design Decision: The DSL is compiled to JSON, not interpreted directly. This decouples the language from the engine. A visual editor, a Python SDK, or an AI story parser all produce the same JSON format. The engine never knows or cares which frontend produced the scene — it only consumes the intermediate representation.

22. Networking

Networking is limited to the orchestration layer. The engine itself has no network dependencies. The network architecture supports:

22.1 Job Submission & Management

The REST API accepts project submissions, returns a job ID, and provides status polling endpoints. Jobs are processed on the server's worker pool. The API design follows OpenCue's job model but simplified for StickFrame's use case.

Endpoints:

- `POST /v1/jobs` — submit a project for rendering
- `GET /v1/jobs/{id}` — get job status, progress, estimated completion
- `GET /v1/jobs/{id}/frames/{range}` — download rendered frames
- `DELETE /v1/jobs/{id}` — cancel a running job
- `GET /v1/jobs/{id}/log` — streaming log output

22.2 Worker Communication

Workers communicate via a message queue. The protocol is:

1. Scheduler publishes frame assignment to queue
2. Worker picks up assignment, downloads scene snapshot
3. Worker renders frames, uploads results
4. Worker publishes completion message
5. Scheduler collects all frames, triggers assembly

WebSocket connections provide real-time progress updates for clients. The progress stream includes per-frame timestamps, worker ID, and any warnings.

23. Security

23.1 Plugin Sandboxing

Plugins execute in a sandboxed environment. The sandbox restricts filesystem access to the project directory, network access (disabled by default), and execution of external binaries (requires manifest permissions). The sandbox is implemented using OS-level process isolation (containers or subprocess with restricted seccomp profiles).

23.2 Scene File Validation

All scene input is validated against a schema before entering the engine. Malformed input is rejected with a machine-readable error. This prevents injection attacks through specially crafted scene files (the asset store checks hash integrity before loading).

23.3 Render Output Integrity

Each frame file includes a content hash. The output assembler verifies that received frames match expected hashes before concatenation. This detects transmission errors and ensures output integrity even with distributed rendering.

23.4 No Execution from Input

The engine never executes code from project files. Actions are defined as data (JSON keyframes), not scripts. Plugin code must be explicitly installed and loaded — a project file cannot introduce new plugin code. This prevents arbitrary code execution through project files.

24. Plugin Architecture

The plugin architecture follows the Microkernel pattern (also called Plugin Architecture). The engine kernel is minimal — it provides scene graph management, timeline evaluation, action player, and rig solver. Everything else is a plugin.

24.1 Plugin Categories

Category	Extension Point	Examples
Renderer	Pipeline stage hook	Software renderer (built-in), GPU renderer, SVG renderer, vector output
Importer	Asset loading hook	.stk (Pivot) importer, Spine JSON importer, DragonBones importer, Blender export import
Exporter	Output format hook	MP4 (built-in), GIF, WebM, PNG sequence, PDF flipbook, API post
Physics	Physics solver hook	Built-in 2D physics, Box2D wrapper, custom solver
Effect	Render stage effect hook	Blur, glow, shadow, chromakey, color grading, watercolor filter
Generator	Asset/action generation hook	Procedural background generator, procedural action generator, TTS voiceover
Analysis	Post-render analysis hook	Scene complexity estimator, render time predictor, visual diff checker

24.2 Plugin Interface

Each plugin is a Python package with a standard entry point. The engine discovers plugins by scanning registered paths (project-local, user-global, system-global). Plugins declare their capabilities in a manifest:

```
manifest = {
    "name": "watercolor_effect",
    "version": "1.0.0",
    "type": "effect",
    "hooks": ["render_stage_4"],
    "dependencies": {"pillow": ">=10.0"},
    "sandbox_permissions": ["filesystem:project_only"],
    "settings": {"brush_size": {"type": "int", "default": 3}}
}
```

The engine calls the plugin at the specified hook point with the current draw command list. The plugin modifies, adds, or removes commands. Multiple plugins can hook the same stage; they execute in priority order (defined in the manifest).

25. API Strategy

The StickFrame API provides programmatic access to the rendering pipeline. It is designed for three use cases: integration with external tools, automated batch rendering, and custom frontend development.

25.1 API Versions

Version	Status	Features
v1 (alpha)	Local Python API	Direct Python imports. <code>from stickframe import Scene, render</code> . Sync API for small jobs.
v2 (beta)	REST API	Async job submission. Progress polling. Result download. Stateless workers.
v3 (stable)	gRPC + REST	gRPC for high-throughput job submission (protocol buffers for scene data). REST for status and management.

25.2 Python SDK

```
from stickframe import Scene, Character, Action, render

scene = Scene(width=1920, height=1080, fps=30)
hero = Character("hero", rig="bipedal")
hero.appearance = {"head_color": "red", "body_color": "blue"}
scene.add_character(hero)

scene.timeline.add_event(0.0, hero, "idle")
scene.timeline.add_event(2.0, hero, "walk", direction="right")
scene.timeline.add_event(4.0, hero, "jump")
```

```
result = render(scene, output="output.mp4")
# result contains: output_path, render_time, frame_count, checksum
```

25.3 gRPC Service Definition (v3)

The Scene and Timeline data structures map to Protocol Buffer messages. The gRPC service defines RPCs for job submission, status streaming, and result retrieval. This enables clients in any language (Go, Rust, TypeScript, C++) to submit rendering jobs.

26. Testing Strategy

26.1 Unit Testing

- **Scene graph operations:** Test entity creation, parent-child relationships, transform resolution, component queries
- **Rig solver:** Test joint position calculation for known angle sets. Compare against pre-computed ground truth.
- **Action player:** Test keyframe interpolation, looping, blending, transition timing
- **Timeline evaluator:** Test event ordering, parallel track resolution, trigger-based events
- **Camera system:** Test viewport transform calculation, target tracking, shake generation
- **Physics solver:** Test gravity, collision, projectile motion against analytical solutions

26.2 Deterministic Rendering Tests

The engine includes a **rendering test suite** that takes known scene files, renders them, and compares output hashes against stored reference hashes. Any change that alters output (intentional or not) is detected. This is the primary regression prevention mechanism.

Test scenes cover: single character, multiple characters, camera movement, action transitions, dialogue rendering, physics interactions, scene transitions, edge cases (empty scene, single frame, maximum characters).

26.3 Performance Tests

Benchmark scenes track per-frame render time, peak memory usage, and total render time. Performance regressions are caught by comparing against baseline measurements. The benchmark suite includes scenes at various complexity levels (1, 10, 50, 200 characters).

26.4 Integration Tests

- Full pipeline test: script → render → verify output hash
- Plugin API test: install mock plugin, verify hook calls

- Import/export round-trip test: scene → export → reimport → verify identity
- Distributed rendering test: render same scene on 1 worker vs 10 workers, verify hashes match
- Cross-platform test: render on Linux, macOS, Windows — verify hashes match

27. Scalability

StickFrame's architecture scales along three axes: project complexity, rendering parallelism, and system deployment.

27.1 Project Complexity Scaling

Metric	Short Clip (30s)	Episode (10min)	Feature (90min)
Frames	900	18,000	162,000
Characters	1-3	5-15	20-100
Scenes	1	10-30	50-200
Timeline events	10-50	200-2,000	2,000-20,000
Asset size	< 10 MB	100-500 MB	1-10 GB
Render time (1 core)	~1 min	~1 hour	~10 hours
Render time (100 cores)	~1 sec	~36 sec	~6 min

27.2 Rendering Parallelism

Frame rendering is embarrassingly parallel. Each frame is independent. With 100 workers, a 90-minute feature (162,000 frames) renders in ~6 minutes (200 frames per worker, ~5-15 ms per frame). The scheduler distributes frames in chunks to minimize queue overhead.

Worker scaling is linear — 2x workers = 2x throughput, limited only by the bottleneck of the output assembler. The assembler batches completed frames for efficient ffmpeg encoding.

27.3 Storage Scaling

The content-addressable asset store deduplicates across projects. For a studio producing 100 projects with shared asset libraries, storage scales logarithmically — common assets (backgrounds, character templates, SFX) are stored once. Only unique assets consume additional space.

28. Risks

Risk	Severity	Probability	Mitigation
Physics determinism across platforms	High	Medium	Fixed-point math library; cross-platform CI with hash verification; physics validation assertions
Pillow rendering bottleneck at high resolution	Medium	Medium	GPU renderer plugin path; resolution-independent draw commands; multi-threaded frame rendering
Timeline complexity for feature-length projects	Medium	Low	Hierarchical timeline (scenes within episodes within acts); lazy loading of scene data
Plugin sandbox escape	High	Low	OS-level process isolation; seccomp profile; filesystem restrictions; manual plugin review process
LLM nondeterminism in story understanding	Medium	High	Validate LLM output against schema; user can save/lock scene description; engine ignores SUL existence
Cache invalidation complexity	Medium	Medium	Explicit dependency tracking per frame; cache key includes engine version; cache miss is always safe
Distributed rendering network bottleneck	Low	Medium	Frame data is small (~200 KB per 720p PNG); scene definitions are tiny (KB); worker pulls once, renders many

29. Research Areas

These areas require investigation before or during implementation. Each has open questions that need experimental validation.

29.1 Animation Quality

- **Inverse kinematics:** Can IK solvers produce natural-looking stickman poses from endpoint targets (e.g., "hand touches point X")? Research FABRIK (Forward And Backward Reaching Inverse Kinematics) algorithm for 2D bone chains.
- **Procedural action generation:** Can walk cycles, run cycles, and other periodic actions be generated procedurally instead of hand-authored? Research Fourier-series-based gait generation.
- **Motion matching:** For Year 3-4, can we use a database of pre-authored motion clips that the engine selects and blends based on context? Research Unreal Engine's motion matching system.
- **Tweening quality:** What is the minimum set of easing functions needed for natural-looking stickman motion? Research animation curves from Spine and DragonBones.

29.2 Story Understanding

- **Story grammar:** Can story structure be formalized as a grammar (e.g., three-act structure, hero's journey) for deterministic story parsing? Research story grammars from computational narratology.
- **Character consistency:** How to maintain character personality across a long timeline without explicit programming? Research personality models for virtual characters.
- **Dialogue-to-action mapping:** Can dialogue lines be automatically mapped to matching character actions? Research gesture generation from text.

29.3 Rendering

- **Software vs GPU:** At what scene complexity does GPU rendering become necessary? What is the cross-over point in terms of characters, effects, and resolution?
- **Style transfer:** Can stickman output be post-processed into different visual styles (pencil sketch, watercolor, pixel art) without re-rendering? Research neural style transfer for line art.
- **Vector output:** Can the renderer output SVG/vector graphics for infinitely scalable animation? Research SVG animation export for stickman scenes.

29.4 Distributed Systems

- **Optimal chunk size:** What is the optimal number of frames per worker assignment? Too few = high queue overhead, too many = load imbalance. Research dynamic chunk sizing.
- **Worker preemption:** How to handle a worker that disconnects mid-job? Current design: reassign its frames. Research checkpointing for long-frame renders.
- **Hybrid cloud/bare-metal scaling:** Can workers dynamically burst to cloud instances when local pool is saturated? Research Kubernetes-based worker autoscaling.

29.5 Physics

- **Deterministic physics across platforms:** Can we achieve byte-identical physics simulation across different CPU architectures? Research fixed-point math vs IEEE 754 compliance.
- **Hybrid animation/physics:** Can a character partially follow animation and partially be physics-driven (e.g., animated upper body, physics-simulated lower body)? Research constrained physics blending.

30. 33. Five-Year Engineering Roadmap

Year 1: Foundation (Months 1-12)

Quarter	Focus	Deliverables
---------	-------	--------------

Q1	Core engine kernel	Scene graph, timeline evaluator, action player, rig solver. Single stickman, 10 actions, script input, MP4 export. Deterministic rendering verified.
Q2	Character system	Multiple characters, appearance system, action blending, action layering. 30 actions. Prop system. Background rendering.
Q3	Camera & effects	Camera system (pan, zoom, follow, shake). Scene transitions. Particle system. Dialogue/subtitle rendering.
Q4	Plugin system v1	Plugin API (render hooks, effect hooks). Plugin sandbox. First plugin marketplace preview. Python SDK v1. API v1 (local).

Year 2: Scale & Distribution (Months 13-24)

Quarter	Focus	Deliverables
Q1	Distributed rendering	Job queue (Redis), worker pool, job scheduler. Output assembler. REST API v2. Render farm deployment guide.
Q2	Physics system	Deterministic 2D physics engine. Collision detection. Ragdoll physics. Hybrid animation/physics characters.
Q3	Import/export ecosystem	Pivot .stk import. Spine JSON import. DragonBones import. GIF/WebM export. Plugin importers/exporters.
Q4	Performance optimization	Frame caching. Region invalidation. Multi-threaded rendering. Vector output (SVG) research. GPU renderer prototype.

Year 3: Intelligence (Months 25-36)

Quarter	Focus	Deliverables
Q1	Story Understanding Layer	LLM-based script parser (Approach A). Scene segmentation. Character extraction. Action identification. Timeline construction from natural language.
Q2	Story grammar engine	Hybrid approach (Approach C). Constraint validator for story consistency. Character arc tracking. Automated pacing.
Q3	Physics & quality improvements	Inverse kinematics (FABRIK). Procedural action generation research. Motion matching prototype. Animation quality benchmarking.
Q4	Collaborative editing	Project synchronization (CRDT-based). Multi-user editing. Comment/annotation system. Version history.

Year 4: Autonomy (Months 37-48)

Quarter	Focus	Deliverables
Q1	AI Director Layer	Agent-based cinematographer. Automated shot selection. Scene pacing optimization. Director's cut overlay system.

Q2	GPU rendering	Production GPU renderer (OpenGL/Vulkan). Drop-in replacement for software renderer. Same draw commands, different backend.
Q3	Style & effects	Style transfer pipeline. Custom visual effects ecosystem. Advanced particle systems. Procedural backgrounds.
Q4	Enterprise features	SSO/SAML integration. Team management. Usage analytics. White-label option. On-prem deployment.

Year 5: Ecosystem (Months 49-60)

Quarter	Focus	Deliverables
Q1	Plugin marketplace	Public plugin registry. Plugin publishing workflow. Revenue sharing. Plugin analytics. Plugin certification program.
Q2	gRPC API & SDK expansion	API v3 (gRPC + streaming). SDKs for TypeScript, Go, Rust. Webhook system for render completion events.
Q3	Integration ecosystem	Blender addon. After Effects plugin. Unity integration. Unreal integration. n8n/node for workflow automation.
Q4	Platform stabilization	Comprehensive documentation. Performance benchmarks published. Certification suite for plugin developers. Platform reaches steady state.

StickFrame Animation Operating System — Engineering Blueprint v2

Prepared by Hermes (Principal Research Engineer) for the Chief Architect | July 2026

Research tools: SearXNG · Open Notebook · Whoogle · Crawl4AI

Reference: Godot Engine architecture, Bevy ECS system, OpenCue render farm, Nucleus VFX plugin system, Spine/DragonBones skeletal animation patterns

CONFIDENTIAL — ENGINEERING INTERNAL